

Universidad Tecnológica de Panamá

Facultad de Ingeniería en Sistemas Computacionales
Departamento de Arquitectura y Redes de Computadoras
Licenciatura en Ciberseguridad

Informe Técnico

Arquitectura, Vulnerabilidades, Controles y Recomendaciones

Curso: Administracion de Seguridad en Entornos Nube

Grupo 1S3141

Profesor: José Moreno

Autores: Cleidys Guzmán, Mónica Pineda, Victor Pineda

Julio de 2026

1. Introducción

El presente informe técnico consolida los resultados del ejercicio semestral realizado sobre AWSGoat, un laboratorio de seguridad intencionalmente vulnerable desplegado sobre Amazon Web Services mediante Terraform. El documento cubre los dos módulos evaluados, un blog serverless (Módulo 1) y una aplicación interna de recursos humanos sobre contenedores (Módulo 2), y presenta, para cada uno: la arquitectura desplegada, las vulnerabilidades explotadas con su clasificación OWASP y el servicio de AWS afectado, el detalle de los controles de remediación y detección aplicados, y un conjunto de recomendaciones generales orientadas a entornos productivos reales.

2. Arquitectura Desplegada

2.1 Módulo 1 — Blog Serverless

El Módulo 1 está compuesto íntegramente por servicios administrados de AWS, sin infraestructura de cómputo tradicional:

Componente	Función dentro de la arquitectura
Amazon API Gateway	Expone los endpoints HTTP de la aplicación (búsqueda de contenido, gestión de usuarios, publicaciones).
AWS Lambda	Implementa la lógica de negocio asociada a cada endpoint, procesando las solicitudes recibidas desde API Gateway.
Amazon DynamoDB	Almacena la información de usuarios, publicaciones y demás datos persistentes de la aplicación.
Amazon S3	Aloja archivos y contenido estático utilizado por el blog.

El flujo de tráfico es: cliente → API Gateway → función Lambda correspondiente → DynamoDB (datos) / S3 (contenido estático).

2.2 Módulo 2 — Aplicación de Recursos Humanos (ECS / ALB / RDS)

El Módulo 2 representa un escenario de infraestructura más tradicional, basado en contenedores sobre instancias EC2:

Componente	Función dentro de la arquitectura
Amazon ECS (EC2 launch type)	Orquesta la aplicación de recursos humanos en contenedores, sobre instancias EC2 administradas mediante un Auto Scaling Group.
Application Load Balancer (ALB)	Publica la aplicación y distribuye las solicitudes HTTP entrantes hacia las tareas del clúster ECS.
Amazon RDS (MySQL)	Base de datos relacional utilizada para el almacenamiento persistente de la información de la aplicación.
AWS Secrets Manager	Almacena de forma segura las credenciales de acceso a la base de datos RDS.
AWS IAM	Administra los roles ecs-instance-role, ecs-task-role y ec2Deployer-role, y las políticas de acceso asociadas a cada uno.
Instance Metadata Service (IMDS)	Servicio local de las instancias EC2 (169.254.169.254) para obtención de credenciales temporales del rol asignado.
AWS Systems Manager (SSM)	Permite la administración remota de instancias EC2 mediante ejecución de comandos, sin requerir acceso SSH directo.

El flujo de tráfico es: Internet → Application Load Balancer → instancias EC2 del Auto Scaling Group → contenedor ECS (aplicación PHP) → Amazon RDS (datos) / AWS Secrets Manager (credenciales de base de datos).

3. Vulnerabilidades Explotadas

3.1 Módulo 1 — Blog Serverless

ID	Vulnerabilidad	Servicio AWS afectado	Clasificación OWASP
R1	Cross-Site Scripting (XSS) reflejado en la barra de búsqueda	Aplicación servida vía API Gateway / Lambda	A03:2021 — Injection
R2	Inyección SQL en el endpoint /v1/search-author	AWS Lambda, Amazon DynamoDB	A03:2021 — Injection
R3	IDOR en el endpoint de cambio de contraseña	AWS Lambda, Amazon DynamoDB	A01:2021 — Broken Access Control

R4	Exposición de datos sensibles vía endpoint oculto /v1/dump	Amazon API Gateway, Amazon DynamoDB	A01:2021 — Broken Access Control
----	--	-------------------------------------	----------------------------------

R1 se explotó mediante el payload `<img src='a' onerror=alert('xss')>`, evadiendo el filtro existente sobre la etiqueta `<script>`. R2 se explotó modificando los parámetros "value" y "authlevel" de la solicitud mediante Burp Suite Repeater, obteniendo el volcado completo de usuarios. R3 se explotó manipulando el parámetro "id" del cuerpo de la solicitud de cambio de contraseña, logrando modificar la contraseña de una cuenta ajena (id=1). R4 se descubrió mediante enumeración de rutas con Burp Intruder sobre el endpoint /v1/list-posts, revelando la ruta no documentada /v1/dump.

3.2 Módulo 2 — Aplicación de Recursos Humanos

#	Vulnerabilidad	Servicio AWS afectado	Clasificación OWASP
1	Inyección SQL en el mecanismo de autenticación	Amazon ECS, Amazon RDS (MySQL)	A03:2021 — Injection
2	Carga insegura de archivos con ejecución remota de código	Amazon ECS, AWS IAM	A05:2021 — Security Misconfiguration
3	Escalación de privilegios local y escape de contenedor	Amazon ECS, Amazon EC2, AWS IAM, IMDS	A01:2021 — Broken Access Control
4	Escalación de privilegios IAM (iam:PassRole, ec2:RunInstances)	AWS IAM, Amazon EC2	A01:2021 — Broken Access Control

La Vulnerabilidad 1 se explotó con el payload `' or '1'='1'#`, combinado con cláusulas LIMIT y ORDER BY DESC para escalar entre roles de usuario hasta Administrador. La Vulnerabilidad 2 aprovechó la ausencia de validación de tipo de archivo en el módulo de carga de Manager para subir un web shell en PHP y obtener credenciales del rol ecs-task-role. La Vulnerabilidad 3 combinó una entrada de sudoers insegura (ejecución de vim como root sin contraseña) con la capability SYS_PTRACE del contenedor para escapar hacia el host EC2 y robar credenciales del rol ecs-instance-role vía IMDS. La Vulnerabilidad 4 encadenó un permission boundary que permitía iam:PassRole y ec2:RunInstances sin

restricción con la existencia de un rol de la cuenta (ec2Deployer-role) con política de acceso total, permitiendo crear un usuario administrador de la cuenta AWS.

4. Detalle de Controles Aplicados

Los controles de remediación y detección documentados a continuación fueron implementados y verificados funcionalmente sobre el Módulo 2, tanto mediante modificación directa del código Terraform del proyecto como mediante configuración de servicios de detección nativos de AWS por consola. Para el Módulo 1 se presentan únicamente recomendaciones (sección 5), dado que no se dispone de evidencia de controles implementados sobre ese módulo específico.

4.1 Corrección de roles y políticas de IAM con permisos excesivos

Problema identificado	Acción realizada	Beneficio de seguridad
El rol ecs-instance-role tenía adjunta la política administrada IAMFullAccess.	Se sustituyó por una política de solo lectura (iam:GetRole, iam:ListRoles, iam:GetInstanceProfile).	El rol ya no puede crear, modificar ni eliminar identidades IAM aunque sea comprometido nuevamente.
El rol ec2Deployer-role tenía una política con Action:* y Resource:* (acceso administrativo total).	Se restringió a permisos exclusivos de gestión de instancias EC2 (RunInstances, TerminateInstances, DescribeInstances, CreateTags).	Elimina la pieza que permitía escalar a AdministratorAccess al final de la cadena de ataque.
El permission boundary de la instancia permitía iam:PassRole sin restricción e incluía iam:PutRole*.	Se eliminó iam:PutRole* y se condicionó iam:PassRole a un único rol destino con la condición iam:PassedToService.	Corta el mecanismo específico usado para lanzar una instancia con un rol de mayor privilegio.
El rol ecs-task-role tenía adjunta la política SecretsManagerReadWrite (lectura y escritura sobre todos los secretos de la cuenta).	Se sustituyó por una política que permite únicamente secretsmanager:GetSecretValue sobre el secreto específico de RDS.	Limita el alcance de un eventual robo de credenciales del contenedor a un único secreto necesario para la operación de la aplicación.

4.2 Corrección de buckets S3 públicos, sin cifrado o sin logging

Problema identificado	Acción realizada	Beneficio de seguridad
El bucket de Terraform state no tenía bloqueo de acceso público, cifrado, versionado ni registro de acceso.	Se habilitaron bloqueo de acceso público, cifrado en reposo (AES256), versionado, y logging de acceso hacia un bucket	Reduce el riesgo de exposición pública accidental y permite auditar quién accede al archivo

	dedicado.	de estado de la infraestructura.
No existía un bucket centralizado para recibir logs de auditoría de la cuenta.	Se creó un bucket dedicado (aws-goat-m2-logs) con bloqueo de acceso público, que recibe los logs de CloudTrail, del ALB y del bucket de state.	Centraliza la evidencia de auditoría y facilita su revisión y retención.

4.3 Habilitación de registros (logging)

Problema identificado	Acción realizada	Beneficio de seguridad
No existía registro de las llamadas a la API de AWS realizadas en la cuenta.	Se habilitó AWS CloudTrail (multi-alcance de la cuenta), entregando eventos tanto a S3 como a un log group de CloudWatch Logs.	Permite reconstruir cronológicamente cualquier acción realizada sobre la cuenta, incluyendo intentos de escalación de privilegios.
No existía registro de las solicitudes HTTP recibidas por el Application Load Balancer.	Se habilitaron los access logs del ALB hacia el bucket de logs centralizado.	Permite identificar patrones de tráfico anómalo o intentos de explotación repetidos contra la aplicación.
Los contenedores de la aplicación no enviaban sus logs a ningún destino centralizado.	Se configuró un log group de CloudWatch (/ecs/aws-goat-m2) y se referenció en la definición de la tarea ECS (logDriver awslogs).	Facilita el diagnóstico y la correlación de eventos a nivel de aplicación con los eventos de infraestructura.

4.4 Configuración de alertas en CloudWatch

Problema identificado	Acción realizada	Beneficio de seguridad
No existía ninguna notificación en tiempo real ante eventos críticos de la cuenta.	Se creó un tema de Amazon SNS con suscripción por correo electrónico, referenciado como acción de las alarmas configuradas.	Convierte cualquier alarma disparada en una notificación efectiva hacia una persona responsable.
No existía forma de detectar en tiempo real la creación de usuarios o políticas administrativas.	Se configuró un Metric Filter sobre el log group de CloudTrail (patrón CreateUser / AttachUserPolicy / CreatePolicy) y una alarma de CloudWatch asociada, con notificación a SNS.	Detecta en minutos el mismo evento que marcó el desenlace de la cadena de ataque explotada (creación del usuario administrador), permitiendo una respuesta temprana.
No existía monitoreo de disponibilidad del servicio	Se configuró una alarma sobre la métrica HealthyHostCount del	Provee visibilidad temprana ante caídas del servicio, ya sea por

expuesto por el ALB.	Target Group, notificando a SNS si no hay hosts sanos.	fallas operativas o por un eventual ataque de denegación de servicio.
----------------------	--	---

El funcionamiento de la alarma de cambios de IAM fue verificado de forma funcional: se ejecutó ``aws iam create-user`` como prueba dirigida, confirmando que el evento fue capturado por CloudTrail, incrementó la métrica personalizada correspondiente, disparó la alarma (cambio de estado a "In alarm") y generó la notificación por correo electrónico esperada.

4.5 Aplicación de tagging y nomenclatura estándar

Se configuró un bloque de etiquetado por defecto (default_tags) a nivel del proveedor de Terraform, aplicando automáticamente las etiquetas Project, Environment, Owner y ManagedBy a todo recurso nuevo creado por el código, sin necesidad de declararlas de forma individual en cada bloque de recurso. Esto garantiza consistencia en el etiquetado de toda la infraestructura y facilita la identificación de propietario y propósito de cada recurso durante una auditoría.

4.6 Controles de detección adicionales (consola de AWS)

Control	Función	Vulnerabilidad relacionada
Amazon GuardDuty	Detección de cambios de política en buckets S3 y actividad anómala en la cuenta.	Refuerza la detección de la cadena de escalación de privilegios (Vulnerabilidad 4).
Amazon Macie	Identificación de objetos con datos sensibles y acceso público indebido en Amazon S3.	Relacionado con la exposición de artefactos sensibles en buckets S3.
AWS Config	Regla administrada iam-policy-no-statements-with-admin-access, que detecta políticas IAM con permisos totales.	Detecta directamente el patrón de política explotado en la Vulnerabilidad 4.
AWS IAM Access Analyzer	Identificación de recursos compartidos fuera de la zona de confianza de la cuenta.	Complementa la detección de configuraciones de acceso indebido.

5. Recomendaciones Generales

5.1 Módulo 1 — Blog Serverless

- Aplicar codificación de salida (output encoding) y una política de Content Security Policy (CSP) para mitigar XSS de forma sistemática.
- Usar consultas parametrizadas o los mecanismos seguros del SDK de DynamoDB, junto con validación de esquema de los parámetros recibidos.
- Vincular toda operación sensible al usuario autenticado en el servidor, sin confiar en identificadores enviados por el cliente; aplicar control de acceso a nivel de objeto.
- Auditar y eliminar rutas de API no documentadas, aplicando autorización uniforme a toda la superficie de la API por defecto.
- Habilitar logging y alertas sobre patrones de acceso anómalos a nivel de API Gateway.

5.2 Módulo 2 — Aplicación de Recursos Humanos

- Aplicar el principio de menor privilegio en todos los roles IAM, evitando políticas con Action:*/Resource:* y restringiendo iam:PassRole a roles y servicios específicos.
- Revisar de forma independiente la política adjunta a un rol y su permission boundary, dado que el permiso efectivo real es la intersección de ambos.
- Migrar a IMDSv2 (HttpTokens: required) en todas las instancias EC2.
- Evitar pid_mode: host en tareas ECS salvo necesidad justificada, y limitar las capabilities de Linux al mínimo indispensable.
- Aplicar validación de tipo de archivo de forma consistente en todos los módulos de carga, sin depender del rol del usuario.
- Mantener activos de forma permanente los controles de detección (CloudTrail hacia CloudWatch Logs, métricas y alarmas sobre eventos críticos de IAM) en cualquier entorno productivo.
- Considerar Service Control Policies (SCPs) a nivel de organización que bloqueen de forma preventiva la creación de usuarios o políticas administrativas fuera de un flujo de aprobación controlado.

5.3 Recomendación transversal

Ambos módulos evidencian que ningún control aislado es suficiente: la defensa efectiva requiere validación en el backend, control de acceso a nivel de objeto, mínimo privilegio en IAM, y monitoreo activo con alertas operando en conjunto. La ausencia de cualquiera de estas capas, incluso si las demás están correctamente implementadas, puede ser suficiente para que una cadena de vulnerabilidades aparentemente menores derive en el compromiso total de un entorno productivo.